

Register Feature Documentation

Contract Table

Field	Contract
Feature	User registration
Endpoint	POST /api/auth/register
Request Body	{ firstName, email, password }
Frontend Validation	Required name/email/password. Password: 8+ chars, one uppercase, one special character.
Backend Validation	Required fields, email format, password policy, duplicate email in auth_user.json and SQLite.
Password Storage	Plaintext is never stored. Server hashes with bcrypt before saving.
JSON Persistence	Append username, bcrypt password hash, firstName, registrationDate to auth_user.json.
Database Persistence	Insert user into SQLite users table with bcrypt hash.
Success Response	201 Created with { message, token, user }
Duplicate Response	409 Conflict with Email is already registered.
Security Notes	JWT carries non-sensitive identity data. Login errors avoid leaking whether email or password failed.

Activity Diagram

```
User opens register.html
-> User enters name, email, and password
-> Frontend validates password policy
-> POST /api/auth/register
-> Backend validates fields and email
-> Backend checks duplicate email in auth_user.json and SQLite
-> bcrypt.hash(password)
-> Append user to auth_user.json
-> Insert user into SQLite
-> jwt.sign({ sub, email }, secret)
-> Return token and safe user profile
-> Frontend stores token and redirects to shop.html
```

Sequence Diagram

```
User -> register.js: Enter name, email, password
register.js -> register.js: Validate uppercase, special char, length
register.js -> Express: POST /api/auth/register
Express -> auth_user.json: Read existing usernames
Express -> SQLite: SELECT user WHERE email = ?
SQLite -> Express: Existing user or null
[Duplicate] Express -> register.js: 409 Conflict
[New] Express -> Express: bcrypt.hash(password)
Express -> auth_user.json: Append hashed user record
Express -> SQLite: INSERT hashed user record
Express -> JWT: Sign token
Express -> register.js: 201 { token, user }
register.js -> Browser: localStorage.setItem(auth_token)
register.js -> User: Redirect to shop.html
```

GenAI Prompt

Act as a Security Engineer and full-stack Node.js developer. Build a user registration feature for an Express and SQLite ecommerce project. Create POST `/api/auth/register`. Request body must include `firstName`, `email`, and `password`. Validate on frontend and backend that password is at least 8 characters and includes one uppercase letter and one special character. Check duplicate username/email against `auth_user.json` and SQLite. Never store plaintext passwords; use `bcrypt`. Append the new user to `auth_user.json` with `username`, `bcrypt password hash`, `firstName`, and `registrationDate`. Insert the same user into SQLite. Sign a JWT after successful registration and return `{ message, token, user }`. Create `register.html` and `js/register.js` so the browser can register, store the JWT in `localStorage`, and redirect to `shop.html`.